

MVC Project: Multilayer Perceptron from Scratch

Khubaib Ahmed and Roll No: 25i-0016

FAST-NUCES, Islamabad

Abstract. This report details the mathematical derivation, theoretical foundation, and practical implementation of a Multilayer Perceptron (MLP) neural network constructed entirely from scratch. To deeply understand the mechanics of deep learning, no external machine learning frameworks were used; the network was built utilizing raw linear algebra and calculus via the NumPy library. The model was trained on the MNIST handwritten digit dataset using Mini-Batch Gradient Descent and backpropagation. Through systematic hyperparameter tuning and optimizer analysis, the model achieved a final test accuracy of 95.54%, demonstrating stable convergence and effective multi-class classification. This report discusses the forward pass mechanics, loss surface optimization, and the practical challenges of scaling matrix operations.

1 Introduction

Neural networks form the backbone of modern artificial intelligence. While high-level libraries abstract away the underlying mathematics, building a Multilayer Perceptron (MLP) from scratch is essential for understanding the core calculus and linear algebra that drive model convergence. The objective of this project is to implement a fully functional MLP, derive the backpropagation gradients for all parameters manually, and scale the implementation to classify the 60,000-image MNIST dataset.

2 Background

2.1 MLP Theory and Activations

An MLP consists of an input layer, one or more hidden layers, and an output layer. Neurons in each layer compute a weighted sum of their inputs, to which a bias is added. To introduce non-linearity, allowing the network to learn complex patterns, an activation function is applied to this weighted sum. This project utilizes the Sigmoid activation function, defined as $\sigma(z) = \frac{1}{1+e^{-z}}$, which maps outputs to a smooth range between 0 and 1.

2.2 Loss Function and Optimizers

To quantify the network’s prediction error, the Mean Squared Error (MSE) loss function is employed. For multi-class classification, the target labels are one-hot encoded. Optimization is performed using Gradient Descent, which updates weights in the opposite direction of the gradient of the loss function. Advanced variants like Nesterov Accelerated Gradient (NAG) improve upon this by calculating gradients at a projected “lookahead” position, applying momentum to navigate flat regions of the loss surface while braking before overshooting the global minimum.

3 Implementation

3.1 Forward Pass and Backpropagation

The forward pass was fully vectorized to handle mini-batches of data simultaneously. For a given layer l , the pre-activation $Z^{(l)}$ and activation $A^{(l)}$ are computed as $Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}$ and $A^{(l)} = \sigma(Z^{(l)})$.

Backpropagation applies the chain rule of calculus to compute the gradient of the loss with respect to each weight and bias. The error term (delta) is propagated backward from the output layer to the first hidden layer, requiring precise matrix transpositions to ensure the dimensionalities align across the 12 primary parameter matrices.

3.2 Training Procedure

The dataset was shuffled at the beginning of each epoch to prevent cyclic memorization. The training loop utilized Mini-Batch Gradient Descent, parsing the training data into discrete chunks to update the weights more frequently per epoch than standard Batch Gradient Descent, significantly accelerating convergence.

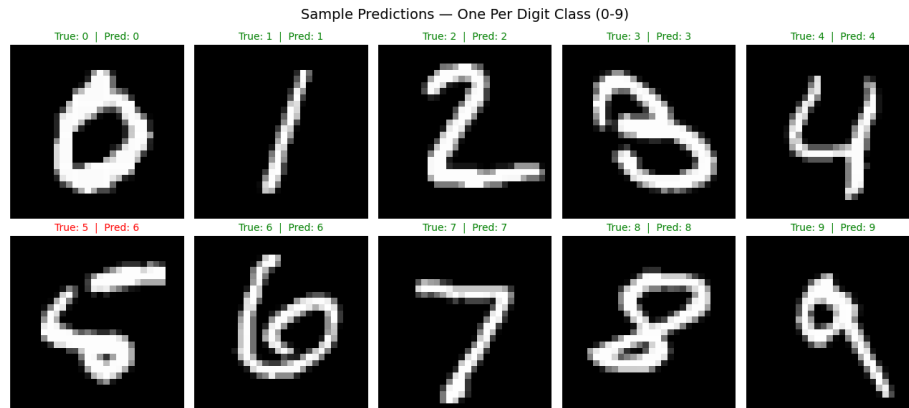
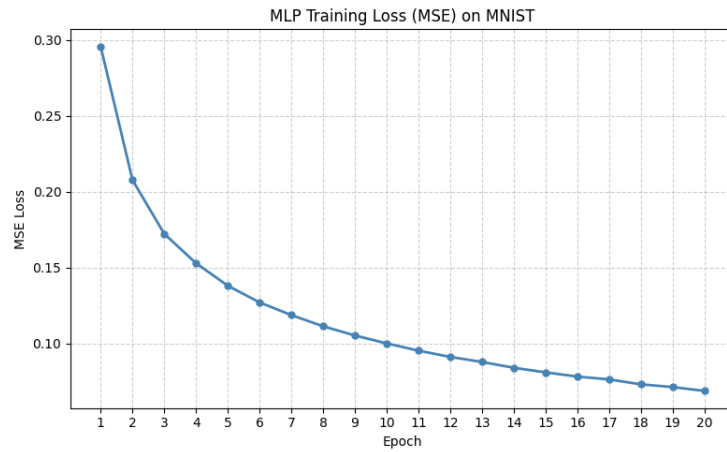
4 Experiments

4.1 Dataset and Hyperparameters

The network was trained on the MNIST dataset, consisting of 60,000 training images and 10,000 test images. Each image is a 28×28 grayscale grid, resulting in an input vector of 784 features. The architecture was defined as $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$. The model was trained for 20 epochs using a batch size of 32 and a learning rate of $\eta = 0.1$.

4.2 Results

The model exhibited excellent learning stability. The MSE loss dropped consistently from an initial value of 0.2957 at Epoch 1 to a final value of 0.0689 at Epoch 20. When evaluated on the unseen 10,000-image test set, the model achieved a classification accuracy of 95.54%.



5 Discussion

The results validate the mathematical correctness of the derived backpropagation equations. The smooth convergence shown in Figure 4.2 indicates that the chosen learning rate and mini-batch size were optimal for the loss landscape. However, scaling to 784 inputs highlights computational bottlenecks; vectorized NumPy operations were strictly necessary to compute the massive matrix multiplications within a reasonable timeframe. Furthermore, using Sigmoid activations in deeper networks introduces a risk of vanishing gradients, which could be mitigated in future iterations by utilizing ReLU activations.

6 Conclusion

This project successfully bridged the gap between theoretical multivariable calculus and practical software engineering. By building an MLP from scratch, deriving its gradients, and successfully classifying the MNIST dataset with 95.54%

accuracy, the foundational mechanics of modern deep learning were practically demonstrated and validated.

References

1. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proceedings of the IEEE* **86**(11), 2278–2324 (1998)
2. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* **323**(6088), 533–536 (1986)

A Appendix A: Optimizer Simulation Working (Task 6C)

To compute the iterations for the optimizers using Mini-Batch Gradient Descent, the following Python script structure was utilized to track velocities, lookahead gradients, and parameter updates.

```
import numpy as np

def plain_gd_update(w, grad, lr):
    return w - lr * grad

def momentum_update(w, v, grad, lr, beta):
    v_new = beta * v + (1 - beta) * grad
    w_new = w - lr * v_new
    return w_new, v_new

def nag_update(w, v, grad_func, lr, beta):
    # Step 1: Lookahead position
    w_lookahead = w - lr * beta * v

    # Step 2: Gradient at lookahead
    g_lookahead = grad_func(w_lookahead)

    # Step 3: Velocity and Weight update
    v_new = beta * v + (1 - beta) * g_lookahead
    w_new = w - lr * v_new
    return w_new, v_new
```